LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7 – ABSTRACT & INTERFACE



FEMBIANA DIKA SAPUTRI

103132400026

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Abstract Class

   Abstract class merupakan kelas yang dideklarasikan menggunakan kata kunci abstract dan tidak dapat dibuat menjadi objek secara langsung. Kelas abstrak dapat memiliki atribut, konstruktor, method biasa, maupun abstract method yang implementasinya diberikan oleh kelas turunannya. Kelas turunan menggunakan keyword extends untuk mewarisi kelas abstrak.

2. Interface

   Interface merupakan kumpulan deklarasi method dan konstanta yang menjadi pedoman atau kontrak bagi kelas yang mengimplementasikannya. Kelas menggunakan keyword ‘implements’ untuk menerapkan interface dan wajib mengimplementasikan method abstrak yang dideklarasikan di dalamnya. Interface juga mendukung pewarisan jamak (multiple inheritance).

3. Default Method dan Static Method

   Pada Java 8, interface dapat memiliki default method dan static method. Default method memiliki implementasi sehingga dapat langsung digunakan oleh kelas yang mengimplementasikan interface. Sementara itu, static method dipanggil melalui nama interface dan tidak perlu diimplementasikan kembali oleh kelas turunannya.

4. Polimorfisme

   Polimorfisme merupakan kemampuan suatu referensi untuk digunakan dalam menangani objek dari kelas yang berbeda. Dengan interface, beberapa objek yang berasal dari hierarki kelas berbeda dapat disimpan dalam satu array bertipe interface dan diproses menggunakan method yang sama.

B. Guided

1. guided.Abstract

   Employee.java

```
package com.fembi.guided.Abstract;

// Fondasi umum: belum punya peran spesifik; jadi tidak boleh diinisiasi langsung
public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
```

```
    }

    // Setiap pegawai pasti punya gaji tapi rumusnya berbeda-beda
    public abstract double earnings();
}
```

Output :

-

SalariedEmployee.java

```
package com.fembi.guided.Abstract;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

Output :

-

Main.java

```
package com.fembi.guided.Abstract;

public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Budi", 5000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" +  salaried.earnings());
    }
}
```

Output :

```
Nama: Budi
Gaji: Rp5000000.0
```

2. guided.Interface

Payable.java

```
package com.febmi.guided.Interface;

public interface Payable {
    double getPaymentAmount();
}
```

Output :

-

Invoice.java

```
package com.febmi.guided.Interface;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    // Hitung tagihan
    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

Output :

-

Employee.java

```
package com.febmi.guided.Interface;

// Fondasi umum: belum punya peran spesifik; jadi tidak boleh diinisiasi langsung
public abstract class Employee implements Payable {
    private String name;

    public Employee(String name){
        this.name = name;
```

```
    }

    public String getName(){
        return name;
    }

    // Setiap pegawai pasti punya gaji tapi rumusnya berbeda-beda
    public abstract double earnings();

    // Saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

Output :

-

SalariedEmployee.java

```
package com.fembi.guided.Interface;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

Output :

-

Main.java

```
package com.fembi.guided.Interface;

public class Main {
    public static void main(String[] args){
        Payable[] items =  new Payable[2];

        items[0] = new Invoice ("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Budi", 5000000);
```

```
        // Dynamic dispatch: Java otomatis mengeksekusi rumus yang tepat sesuai
objek
        for (Payable item : items){
            System.out.println("jumlah yang perlu dibayar: Rp" +
item.getPaymentAmount());
        }
    }
}
```

Output :

```
jumlah yang perlu dibayar: Rp3000000.0
jumlah yang perlu dibayar: Rp5000000.0
```

3. guided.Default_Method_dan_Static_Method_pada_Interface

Payable.java

```
package com.febmi.guided.Default_Method_dan_Static_Method_pada_Interface;

public interface Payable {
    double getPaymentAmount();

    // Fitur baru
    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }

    // Fungsi utilitas
    static void logTransaction(String message){
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

Output :

-

Invoice.java

```
package com.febmi.guided.Default_Method_dan_Static_Method_pada_Interface;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }
```

```
    public String getPartName(){
        return partName;
    }

    // Hitung tagihan
    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

Output :

-

Employee.java

```
package com.febmi.guided.Default_Method_dan_Static_Method_pada_Interface;

// Fondasi umum: belum punya peran spesifik; jadi tidak boleh diinisiasi langsung
public abstract class Employee implements Payable {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // Setiap pegawai pasti punya gaji tapi rumusnya berbeda-beda
    public abstract double earnings();

    // Saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

Output :

-

SalariedEmployee.java

```
package com.febmi.guided.Default_Method_dan_Static_Method_pada_Interface;

public class SalariedEmployee extends Employee {
    private double weeklySalary;
```

```
    public SalariedEmployee(String name, double weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

Output :

-

Main.java

```
package com.febmi.guided.Default_Method_dan_Static_Method_pada_Interface;

public class Main {
    public static void main(String[] args){
        Payable.logTransaction("Sistem pembayaran diinisiasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items){
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi berhasil diproses.");
    }
}
```

Output :

```
[AUDIT LOG] Sistem pembayaran diinisiasi.
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Transaksi berhasil diproses.
```

C. Unguided

Soal — Sistem Kasir Terpadu Bioskop (*Cinema POS*)

Sebuah bioskop memerlukan sistem kasir yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli Tiket Nonton, tetapi juga bisa membeli makanan/minuman (Paket Snack F&B).

Secara konsep sistem, Tiket Bioskop dan Paket Snack berasal dari silsilah keluarga (hierarki) yang sepenuhnya berbeda. Tiket film murni pun bersifat abstrak (tidak bisa dibeli tanpa jenis studio). Namun, di mata program kasir, tiket dan makanan harus bisa disatukan, dihitung total tagihannya, dan dicetak di dalam satu struk secara otomatis.

Pengerjaan soal ini dibagi menjadi 4 tugas utama untuk melatih abstraksi kelas, antarmuka (*interface*), dan fitur modern Java 8+.

1. Tugas 1: Fondasi Kelas Abstrak (*Abstract Class*)

   Buatlah hierarki kelas tiket bioskop dengan spesifikasi berikut:

   1) Kelas Induk TiketBioskop (Wajib abstract)
      - Atribut (Private): judulFilm (String), hargaDasar (double), jumlahKursi (int).
      - Konstruktor: Menerima dan menginisialisasi ketiga atribut tersebut.
      - Method: Buat *getter* untuk ketiga atribut.
      - Abstract Method:
        - public abstract double hitungTotalBayar();
        - public abstract String getNamaTransaksi(); *(Beban perumusan kedua metode ini diserahkan kepada kelas turunannya agar output di struk nanti spesifik untuk tiap jenis tiket).*

   2) Kelas Turunan TiketReguler
      - Atribut Tambahan (Private): nomorStudio (int).
      - Konstruktor: Menginisialisasi atribut induk via super(judulFilm, hargaDasar, jumlahKursi) dan atribut lokal nomorStudio.
      - Override Method:
        - hitungTotalBayar(): Mengembalikan hasil hargaDasar * jumlahKursi.
        - getNamaTransaksi(): Mengembalikan string "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")".

3) Kelas Turunan TiketVIP
    - Atribut Tambahan (Private): biayaFasilitasVIP (double). *(Catatan: Biaya fasilitas seperti sofa recliner ini dikenakan untuk setiap kursi).*
    - Konstruktor: Menginisialisasi atribut induk via super(...) dan atribut lokal biayaFasilitasVIP.
    - Override Method:
        - hitungTotalBayar(): Mengembalikan hasil (hargaDasar + biayaFasilitasVIP) * jumlahKursi.
        - getNamaTransaksi(): Mengembalikan string "Tiket VIP Premiere - " + getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) + ")".

Jawaban :

TiketBioskop.java

```
package com.fembi.unguided;

public abstract class TiketBioskop {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    public abstract String getNamaTransaksi();
}
```

Output :

-

TiketReguler.java

```
package com.fembi.unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar,
                        int jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - "
                + getJudulFilm() + " (" + getJumlahKursi()
                + " x Rp" + getHargaDasar() + ")";
    }
}
```

Output :

-

TiketVIP.java

```
package com.fembi.unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar,
                    int jumlahKursi, double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP)
                * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm()
```

```
                + " (" + getJumlahKursi() + " x Rp"
                + (getHargaDasar() + biayaFasilitasVIP) + ")";
        }
    }
```

Output :
-

Penjelasan :
Kelas TiketBioskop dibuat sebagai abstract class yang menyimpan atribut umum tiket, yaitu judul film, harga dasar, dan jumlah kursi. Atribut tersebut dibuat private dan diakses melalui getter. Kelas ini juga memiliki dua abstract method, yaitu hitungTotalBayar() dan getNamaTransaksi(), yang implementasinya diserahkan kepada kelas turunannya. Kelas TiketReguler menghitung pembayaran berdasarkan harga dasar dikalikan jumlah kursi, sedangkan TiketVIP menghitung pembayaran dengan menambahkan biaya fasilitas VIP pada harga dasar setiap kursi.

2. Tugas 2: Kontrak Antarmuka Lintas Entitas (*Interface*)

Sistem kasir membutuhkan standar publik agar setiap barang bisa diproses.

1) Antarmuka ItemTransaksi Buatlah *interface* bernama ItemTransaksi dengan dua deklarasi kontrak (metode abstrak):
    - double getNominalBayar();
    - String getNamaTransaksi();

2) Integrasi Kelas TiketBioskop ke Antarmuka
    - Tambahkan implements ItemTransaksi pada deklarasi kelas TiketBioskop.
    - Penting (Alasan Arsitektural): Metode hitungTotalBayar() adalah logika internal spesifik tiket, sedangkan getNominalBayar() adalah kontrak publik kasir. Wujudkan metode getNominalBayar() di dalam TiketBioskop yang isinya sekadar me-*return* nilai dari hitungTotalBayar().
    - *(Catatan: Anda tidak perlu menata ulang getNamaTransaksi() di sini karena ia sudah dideklarasikan sebagai metode abstrak pada Tugas 1).*

3) Kelas Baru: PaketSnack (Benda Non-Tiket) Buat kelas independen bernama PaketSnack yang bukan turunan TiketBioskop, tetapi mengimplementasikan ItemTransaksi.
    - Atribut (Private): namaPaket (String), jumlahPorsi (int), hargaPerPorsi (double).
    - Konstruktor: Menginisialisasi ketiga atribut.

- Override Method:
    - getNominalBayar(): Mengembalikan hasil jumlahPorsi * hargaPerPorsi.
    - getNamaTransaksi(): Mengembalikan string "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi + ")".

Jawaban :

ItemTransaksi.java

```
package com.febi.unguided;

public interface ItemTransaksi {
   double getNominalBayar();
   String getNamaTransaksi();
}
```

Output :

-

Memperbarui TiketBioskop.java

```
package com.febi.unguided;

public abstract class TiketBioskop implements ItemTransaksi {
   private String judulFilm;
   private double hargaDasar;
   private int jumlahKursi;

   public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
      this.judulFilm = judulFilm;
      this.hargaDasar = hargaDasar;
      this.jumlahKursi = jumlahKursi;
   }

   public String getJudulFilm() {
      return judulFilm;
   }

   public double getHargaDasar() {
      return hargaDasar;
   }

   public int getJumlahKursi() {
      return jumlahKursi;
   }

   public abstract double hitungTotalBayar();

   public abstract String getNamaTransaksi();
```

```
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

Output :
-

PaketSnack.java

```
package com.fembi.unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi,
                      double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " ("
                + jumlahPorsi + " x Rp" + hargaPerPorsi + ")";
    }
}
```

Output :
-

Penjelasan :
Interface ItemTransaksi dibuat sebagai kontrak yang menetapkan dua method, yaitu getNominalBayar() dan getNamaTransaksi(). Selanjutnya, kelas TiketBioskop dari Tugas 1 diperbarui dengan menambahkan implements ItemTransaksi. Method getNominalBayar() dibuat untuk mengembalikan hasil perhitungan dari hitungTotalBayar(). Selain itu, dibuat kelas PaketSnack yang mengimplementasikan interface ItemTransaksi secara langsung tanpa mewarisi kelas TiketBioskop. Dengan

demikian, tiket bioskop dan paket snack memiliki standar method yang sama sehingga dapat diproses oleh sistem kasir.

3. Tugas 3: Modernisasi Antarmuka Java 8+

   Suntikkan fitur tambahan ke antarmuka ItemTransaksi tanpa mengharuskan kelas-kelas di atas dimodifikasi ulang:

   1) Default Method Tambahkan metode default void cetakRincianItem() yang bertugas mencetak format baris struk ke layar. Panggil metode milik antarmuka (getNamaTransaksi dan getNominalBayar) untuk mengambil data secara dinamis. Format wajib: - [Nama Transaksi (Kuantitas x Harga Satuan)] | Total: Rp[Nominal Bayar]
   2) Static Method Tambahkan metode static void catatLogKasir(String pesan) yang mencetak teks: [LOG SISTEM KASIR] [pesan]

   Jawaban :

   Memperbarui ItemTransaksi.java

```
package com.fembi.unguided;

public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();

    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi()
                + " | Total: Rp" + getNominalBayar());
    }

    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

   Output :

   -

   Penjelasan :

   Interface ItemTransaksi diperbarui dengan menambahkan dua method. Method cetakRincianItem() merupakan default method yang digunakan untuk mencetak nama transaksi dan nominal pembayaran secara otomatis dengan memanfaatkan method yang sudah tersedia pada interface. Sementara itu, method catatLogKasir(String pesan) merupakan static method yang digunakan untuk menampilkan informasi

aktivitas kasir. Penambahan kedua method ini tidak mengharuskan kelas TiketReguler, TiketVIP, dan PaketSnack mengimplementasikannya kembali.

4. Tugas 4: Pengujian Polimorfisme di Main

   Tuliskan alur berikut di dalam *method* main:

   1) Uji Proteksi Abstraksi: Tuliskan baris kode instansiasi TiketBioskop (misal: new TiketBioskop(...)), lalu jadikan baris tersebut sebagai komentar (//).
   2) Buka Kasir: Panggil metode statis catatLogKasir("Membuka transaksi baru untuk Pelanggan #042."); langsung melalui nama antarmukanya.
   3) Buat Objek Transaksi:
      - 1 objek TiketReguler (Film "Interstellar", harga dasar 40000, 2 kursi, Studio 3).
      - 1 objek TiketVIP (Film "Dune: Part Two", harga dasar 60000, 2 kursi, biaya VIP 25000).
      - 1 objek PaketSnack ("Combo Popcorn Caramel + 2 Soda", 1 porsi, harga 55000).
   4) Array Polimorfik: Simpan ketiga objek tersebut ke dalam satu array heterogen bertipe ItemTransaksi[].
   5) Pemrosesan Dinamis (Loop):
      - Gunakan perulangan (*for* atau *for-each*) terhadap array tersebut.
      - Di dalam perulangan, panggil *default method* cetakRincianItem() lewat masing-masing objek/elemen array.
      - Hitung dan jumlahkan nominal bayar keseluruhan belanjaan secara akumulatif.
   6) Tutup Kasir: Tampilkan grand total seluruh belanjaan, lalu panggil kembali metode statis catatLogKasir("Transaksi #042 selesai diproses.");.

Jawaban :

Main.java

```
package com.febi.unguided;

public class Main {
    public static void main(String[] args) {

        // Uji proteksi abstract class
        // TiketBioskop tiket = new TiketBioskop("Film", 40000, 2);

        // Membuka kasir
        ItemTransaksi.catatLogKasir(
                "Membuka transaksi baru untuk Pelanggan #042.");

        // Membuat objek transaksi
        TiketReguler reguler = new TiketReguler(
                "Interstellar", 40000, 2, 3);
```

```
            TiketVIP vip = new TiketVIP(
                  "Dune: Part Two", 60000, 2, 25000);

            PaketSnack snack = new PaketSnack(
                  "Combo Popcorn Caramel + 2 Soda", 1, 55000);

            // Menyimpan objek dalam array polimorfik
            ItemTransaksi[] transaksi = {reguler, vip, snack};

            double totalTransaksi = 0;

            System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");

            // Memproses setiap transaksi
            for (ItemTransaksi item : transaksi) {
               item.cetakRincianItem();
               totalTransaksi += item.getNominalBayar();
            }

            // Menampilkan total pembayaran
            System.out.println("------------------------------------------------------------");
            System.out.println("TOTAL TRANSAKSI : Rp" + totalTransaksi);

      System.out.println("============================================
      ================");

            // Menutup kasir
            ItemTransaksi.catatLogKasir(
                  "Transaksi #042 selesai diproses.");
         }
      }
```

Output :

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Penjelasan :

Kelas Main digunakan untuk menjalankan dan menguji seluruh kelas yang telah dibuat.

1. Uji proteksi abstraksi: Instansiasi TiketBioskop dituliskan sebagai komentar karena abstract class tidak dapat dibuat menjadi objek secara langsung.

2. Membuka kasir: Method catatLogKasir() dipanggil melalui nama interface ItemTransaksi karena merupakan static method.
3. Membuat objek transaksi: Program membuat tiga objek, yaitu TiketReguler, TiketVIP, dan PaketSnack.
4. Array polimorfik: Ketiga objek dimasukkan ke dalam satu array bertipe ItemTransaksi[], meskipun berasal dari kelas yang berbeda.
5. Pemrosesan dinamis: Perulangan for-each digunakan untuk memanggil cetakRincianItem() dan menjumlahkan seluruh nominal pembayaran.
6. Menutup kasir: Program menampilkan total transaksi dan mencatat bahwa transaksi telah selesai diproses.

Penggunaan array ItemTransaksi[] menunjukkan penerapan polimorfisme karena satu tipe referensi dapat menangani beberapa objek berbeda.

D. Kesimpulan

Berdasarkan praktikum Modul 7 tentang Abstract dan Interface, dapat disimpulkan bahwa abstract class digunakan sebagai kelas induk yang menyediakan struktur umum bagi kelas turunannya, sedangkan interface digunakan untuk menetapkan kontrak method yang harus dimiliki oleh kelas yang mengimplementasikannya.

Penerapan keduanya pada sistem kasir bioskop memungkinkan objek tiket reguler, tiket VIP, dan paket snack diproses dalam satu sistem meskipun memiliki struktur kelas yang berbeda. Penggunaan default method dan static method juga membantu menyediakan fungsi tambahan pada interface. Selain itu, penerapan polimorfisme melalui array `ItemTransaksi[]` memudahkan program dalam menampilkan rincian dan menghitung total pembayaran secara otomatis.

Dengan demikian, praktikum ini memberikan pemahaman mengenai penerapan abstract class, interface, enkapsulasi, inheritance, dan polimorfisme dalam membangun program Java yang lebih terstruktur.